B.Tech. Cloud / Cloud Native Computing Laboratory

LABORATORY MANUAL

Implement Service Discovery and Load Balancing in Kubernetes

Item	Details
Programme	B.Tech.
Suggested Duration	2 Hours
Experiment Type	Hands-on / Individual or Pair
Platform	Docker Desktop with built-in Kubernetes (recommended)
Alternative	Minikube / local Kubernetes cluster
Difficulty	Intermediate

Faculty Note: Students should work with a local Kubernetes cluster and should verify that kubectl is connected to the intended context before creating resources.

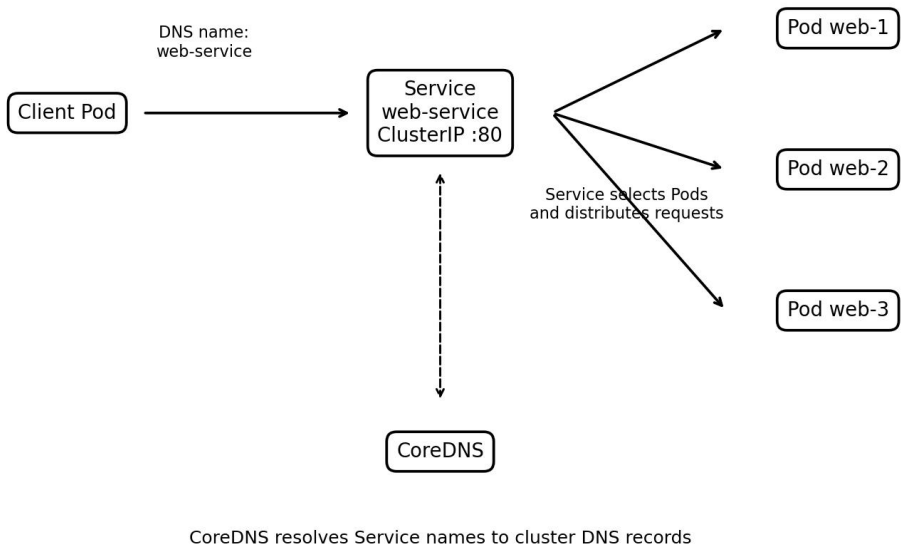


Figure 1. Service discovery and load-balancing flow used in this experiment.

1. Experiment Overview

Kubernetes uses a Service as a stable abstraction in front of a changing set of Pods. The Service provides a consistent endpoint while Pods can be created, deleted, and replaced. Inside the cluster, Kubernetes DNS provides a stable name for the Service, allowing applications to discover a backend without hard-coding individual Pod IP addresses. Traffic sent to a Service is then routed to the selected backend endpoints.

Aspect	What students will demonstrate
Service discovery	Resolve a Kubernetes Service by its DNS name from another Pod.
Stable endpoint	Use the Service name instead of a Pod IP address.
Load balancing	Send repeated HTTP requests to one Service and observe different backend Pods.
Scaling	Increase or decrease Deployment replicas and observe the Service endpoints change.
Troubleshooting	Use kubectl to diagnose selector, endpoint, DNS, and readiness problems.

2. Learning Objectives

- Explain the role of a Kubernetes Service in service discovery.
- Differentiate a Pod IP from a Service IP and Service DNS name.
- Create a Deployment with multiple replicas and expose it through a ClusterIP Service.
- Verify Service discovery using Kubernetes DNS from a client Pod.
- Demonstrate request distribution across multiple Pods.
- Inspect Endpoints/EndpointSlices and relate them to Service selectors.
- Scale the workload and verify that the Service automatically tracks the new replicas.

3. Prerequisites

- Basic understanding of containers, Docker images, Pods, Deployments, and YAML.
- Windows / Linux / macOS computer with Docker Desktop installed and Kubernetes enabled, OR a working Minikube/local Kubernetes cluster.
- kubectl installed and available in the terminal.
- At least 4 GB RAM available for Docker/Kubernetes during the lab.
- Internet access for the first pull of the container images.

Important: Run all kubectl commands against the same cluster. In Docker Desktop, verify the current context with kubectl config current-context. In Minikube, use minikube status.

4. Concept Review

4.1 Pod IP vs Service IP

A Pod receives an IP address, but a Pod is not a reliable application endpoint because Pods can be recreated. A Service provides a stable virtual endpoint and selects backend Pods using labels. For this experiment, the Service will select all Pods whose label includes app=web.

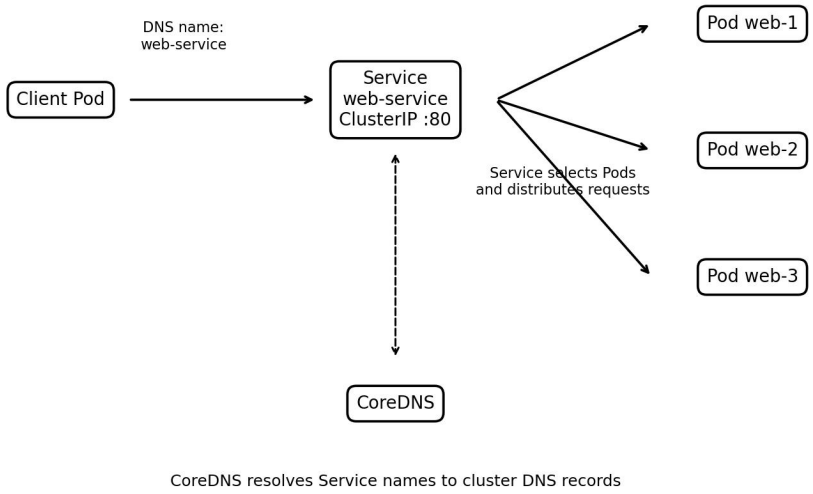
4.2 Service Discovery

Kubernetes creates DNS records for Services. A client in the same namespace can normally use the short Service name, for example web-service. A fully qualified form is web-service.default.svc.cluster.local.

4.3 Load Balancing

The Service sends traffic to its selected endpoints. The exact routing mechanism depends on the cluster networking/service-proxy implementation, so students should focus on the observable behavior: requests to the stable Service endpoint are delivered to backend Pods rather than requiring the client to track Pod IPs manually.

5. Lab Architecture



Experiment flow: Client Pod -> DNS lookup of web-service -> Service ClusterIP -> one of the selected web Pods.

6. Required Kubernetes Objects

Object	Name	Purpose
Deployment	web-deployment	Creates and manages three web Pods.
Service	web-service	Provides stable discovery and load-balancing endpoint.
Pod (temporary)	client	Tests DNS resolution and sends HTTP requests.

7. Procedure

Step 1 - Verify the Kubernetes Cluster

1. Open PowerShell/Command Prompt/terminal.
2. Verify kubectl is available.

```
kubectl version --client
kubectl config current-context
kubectl get nodes
```

Expected result: At least one node should be Ready. Record the context name used in your laboratory notebook.

Step 2 - Create a Working Namespace (Recommended)

```
kubectl create namespace service-lab
kubectl config set-context --current --namespace=service-lab
```

Verify:

```
kubectl get all
```

Step 3 - Create the Multi-Replica Web Deployment

Create a file named web-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:alpine
          command: ["/bin/sh", "-c"]
          args:
            - |
              echo "Hello from Kubernetes Pod: $(hostname)" > /usr/share/nginx/html/index.html
              nginx -g 'daemon off;'
          ports:
            - containerPort: 80
```

Why the hostname is displayed: The page content includes the Pod hostname. When repeated requests reach different replicas, students can see which backend Pod answered.

Apply the Deployment:

```
kubectl apply -f web-deployment.yaml
kubectl get deployments
kubectl get pods -l app=web -o wide
```

Step 4 - Observe Pod Labels and IP Addresses

```
kubectl get pods --show-labels -o wide
```

Record the three Pod names and their IP addresses. These are backend endpoints selected by the Service. Do not use these Pod IPs as the application endpoint for later testing; the purpose of the Service is to avoid that dependency.

Step 5 - Create the Kubernetes Service

Create a file named web-service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

Apply the Service:

```
kubectl apply -f web-service.yaml
kubectl get service web-service
```

Inspect the Service in more detail:

```
kubectl describe service web-service
```

Step 6 - Inspect Service Endpoints

```
kubectl get endpoints web-service
kubectl get endpointslices -l kubernetes.io/service-name=web-service
```

You should see the backend addresses corresponding to the selected Pods. The Service selector app=web is responsible for finding these Pods.

Step 7 - Demonstrate Service Discovery Using Kubernetes DNS

Create a temporary client Pod:

```
kubectl run client --image=busybox:1.36 --restart=Never -- sleep 3600
```

Verify it is running:

```
kubectl get pod client
```

Enter the client container:

```
kubectl exec -it client -- sh
```

Inside the client Pod, run:

```
nslookup web-service
wget -qO- http://web-service
```

The DNS query should resolve the Service name. The HTTP request should return the page generated by one of the web Pods.

Test the namespace-qualified name:

```
wget -qO- http://web-service.service-lab.svc.cluster.local
```

Exit the client shell:

```
exit
```

Step 8 - Demonstrate Load Balancing

Run several HTTP requests from the client Pod. Repeat enough times to observe that requests may be served by different Pods.

```
kubectl exec client -- sh -c 'for i in 1 2 3 4 5 6 7 8 9 10; do wget -qO- http://web-service;
echo; done'
```

Observation: The output should contain Pod hostnames. The exact order and distribution are implementation-dependent; the key observation is that the client talks to the stable Service endpoint and does not address individual Pod IPs.

Step 9 - Scale the Application

```
kubectl scale deployment web-deployment --replicas=5
kubectl get pods -l app=web
kubectl get endpointslices -l kubernetes.io/service-name=web-service
```

Repeat the request test:

```
kubectl exec client -- sh -c 'for i in 1 2 3 4 5 6 7 8 9 10; do wget -qO- http://web-service;
echo; done'
```

Compare the set of backend Pod hostnames before and after scaling. The Service should automatically use the newly available Pods once they become ready.

Step 10 - Demonstrate the Effect of a Selector Error

This step is for troubleshooting practice. Do not change the Deployment selector. Instead, temporarily edit the Service selector so it does not match any Pod, then restore it.

```
kubectl edit service web-service
```

Change:

```
selector:
  app: web
```

to:

```
selector:
  app: does-not-exist
```

Then inspect:

```
kubectl get endpoints web-service
kubectl describe service web-service
```

Expected diagnostic: The Service should have no matching endpoints. HTTP requests to the Service will fail because no backend Pod matches the selector. Restore app: web and verify the endpoints return.

Step 11 - Clean Up

```
kubectl delete pod client
kubectl delete service web-service
kubectl delete deployment web-deployment
kubectl config set-context --current --namespace=default
kubectl delete namespace service-lab
```

Cleanup rule: Always remove test resources after the experiment, especially when working on shared clusters.

8. Observation Sheet

Activity	What to record	Expected observation
Cluster verification	Context + node status	Node(s) are Ready.
Deployment	Replica count + Pod names	Three web Pods created initially.
Service	ClusterIP + port	Stable ClusterIP for web-service.
Endpoints	Endpoint addresses	Multiple backend addresses are attached to the Service.
DNS test	DNS result for web-service	Service name resolves inside the cluster.
Request test	10 response hostnames	Responses come through the Service and may show multiple backend Pods.
Scale test	3 replicas -> 5 replicas	New Pods become Service endpoints when ready.
Selector failure	Endpoint count after mismatch	No matching endpoints.

9. Understanding the YAML

Field	Meaning in this experiment
Deployment.spec.replicas	Desired number of web Pod replicas.
matchLabels: app=web	Deployment manages Pods having this label.
template.metadata.labels	Label assigned to created Pods.
Service.spec.selector	Selects the Pods that receive Service traffic.
port: 80	Port exposed by the Service.
targetPort: 80	Port on the selected Pod.
type: ClusterIP	Service is reachable internally inside the cluster.
containerPort: 80	Documents the port used by the web container.

10. Troubleshooting Guide

Problem	Useful command	Likely cause / action
Pod is Pending	kubectl describe pod <pod>	Insufficient resources, scheduling, or image pull issue.
Pod is CrashLoopBackOff	kubectl logs <pod>	Inspect container command and application logs.
Service has no endpoints	kubectl get pods --show-labels; kubectl describe svc web-service	Service selector does not match Pod labels or Pods are not Ready.
DNS lookup fails	kubectl exec client -- nslookup web-service	Check client Pod, CoreDNS health, and namespace/context.
HTTP request fails	kubectl exec client -- wget -qO- http://web-service	Check Service endpoints and targetPort.
Wrong namespace	kubectl config view --minify | findstr namespace	Ensure commands and resources are in service-lab.
No image pull	kubectl describe pod <pod>	Check internet access, image name, and registry availability.

13. Experiment Deliverables / Student Submission

- Objective and brief theory.
- Both YAML files used in the experiment.
- Screenshot of kubectl get nodes.
- Screenshot of three initial web Pods.
- Screenshot of Service with ClusterIP.
- Screenshot of Endpoints/EndpointSlices.
- Screenshot/output of nslookup web-service.
- Output showing multiple backend hostnames from repeated requests.
- Evidence of scaling from 3 to 5 replicas.
- Observation and conclusion.
- Answers to the analysis questions and viva questions.

15. Expected Learning Outcome

After completing this experiment, a student should be able to deploy a multi-replica workload, create a Kubernetes Service, discover the service through cluster DNS, observe traffic reaching multiple backend Pods, inspect Service endpoints, and explain how Kubernetes decouples service consumers from changing Pod IP addresses.

16. Extension Activities

- Change the replica count from 3 to 1 and repeat the request test. Compare the observed backend hostnames.
- Create a second Service with NodePort and compare internal ClusterIP access with host-facing access.
- Create a second namespace and deploy another Service with the same name. Test why short DNS names are namespace-scoped.
- Modify the web content so each replica displays a different application version label.
- Compare a normal Service with a headless Service (clusterIP: None) and inspect DNS results.
- Introduce readiness probes and observe how endpoint membership changes when a Pod is not ready.

18. References

- Kubernetes Documentation - Service: https://kubernetes.io/docs/concepts/services-networking/service/
- Kubernetes Documentation - DNS for Services and Pods: https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/

- Kubernetes Documentation - Services, Load Balancing, and Networking: https://kubernetes.io/docs/concepts/services-networking/
- Kubernetes Documentation - Access Applications in a Cluster: https://kubernetes.io/docs/tasks/access-application-cluster/